HW1 – Group39

Turboult Olivier (s366835) **Del Coco Matej** (s362121) **Aimone Daniele** (s348752)

1. Results

Kernel	OI (FLOP/byte)	Median latency (ms)	Std. latency (ms)
Dense scalar	42.7	592.3	12.0
Dense tiled with NEON and OpenMP	42.7	81.1	3.7
Sparse scalar	20.9	97.7	1.4
Sparse NEON	20.9	40.4	1.3
Sparse tiled	20.9	26.4	0.0
Sparse OpenMP (2 threads)	20.9	42.9	1.0
Sparse OpenMP (4 threads)	20.9	21.9	0.4
Sparse Combined (2 threads)	20.9	14.6	0.1
Sparse Combined (4 threads)	20.9	7.3	0.0

These latencies are measured on a Raspberry Pi 4, we compute two different benchmark runs. Median = average of the 2 benchmark run medians; Std = standard deviation between the medians of the 2 benchmark runs.

Operational intensity

Assumptions. OI = FLOP/bytes. Ideal model: each value is moved once (single memory level, infinite registers). 1 MAC = 2 FLOP; 1 float or index = 4 B. $M = 128$, $K = N = 512$, $nnzb = 4096$.

Dense kernels

$$\text{FLOP} = 2MKN = 2 \cdot 128 \cdot 512 \cdot 512 = 67{,}108{,}864$$

$$\text{Bytes} = 4\,(\underbrace{MK}_{A} + \underbrace{KN}_{W} + \underbrace{MN}_{C}) = 4\,(65{,}536 + 262{,}144 + 65{,}536) = 1{,}572{,}864$$

$$\text{OI} = 67{,}108{,}864/1{,}572{,}864 = 42.7 \text{ FLOP/B}$$

The tiled NEON/OpenMP version does the same FLOPs on the same data, so it's OI is identical.

Sparse kernels

Each of the 128 input rows does 16 MACs per stored block:

$$\text{FLOP} = 2 \cdot M \cdot 16 \cdot nnzb = 2 \cdot 128 \cdot 16 \cdot 4096 = 16{,}777{,}216$$

$$\text{Bytes} = 4\,(\underbrace{MK}_{A} + \underbrace{16\,nnzb}_{\text{values}} + \underbrace{N/4 + 1}_{\text{col_ptr}} + \underbrace{nnzb}_{\text{row_idx}} + \underbrace{MN}_{C})$$

$$= 262{,}144 + 262{,}144 + 516 + 16{,}384 + 262{,}144 = 803{,}332$$

$$\text{OI} = 16{,}777{,}216/803{,}332 = 20.9 \text{ FLOP/B} \quad (21.3 \text{ without index arrays})$$

NEON, tiling and OpenMP change the instructions, the order and the thread doing each MAC, but not the total FLOPs or the data moved, so all sparse kernels share OI = 20.9.

Dense vs. sparse. Sparsity cuts FLOPs by 4× but bytes only by ≈ 2× (A and C are unchanged), so the OI halves.